实验

让车辆看前方图像，根据当前速度决定加速或制动，在靠近障碍物时把速度降下来。

整个框架只有三个核心职责：

- 感知前端：把图片变成控制器可以使用的信息。
- 控制器：根据信息决定动作。
- 安全分析：检查这个控制器是否会把系统带入危险状态。

图像-->语义表示-->控制器的过程

当前完整顺序是：

```
一帧RGB图像
    → 图像预处理
    → 冻结DINO提取384维视觉特征
    → 特征标准化和投影网络
    → 32维任务语义表示，也就是图像latent
    → 加上当前速度，组成33维观测
    → 训练好的PPO动作网络
    → 加速或制动指令
```

图像预处理负责把图片整理成视觉网络接受的格式。当前保持RGB通道，送入DINO前调整到224×224，并按固定规则缩放像素数值。这一步没有预测距离，也没有决定动作。

DINO提取视觉特征，将图片变成384个数字。这些数字是网络从画面中提取的整体视觉信息，不是384个已经标注好的物体名称或距离。DINO主体保持冻结，沿用预训练的视觉能力。

投影网络产生任务语义表示。DINO的通用特征先使用已保存的训练统计量进行标准化，避免不同特征的数值尺度相差过大；再由投影网络转换为32个数字。投影训练希望保留与当前制动任务有关的信息，使同距离不同天气、颜色的表示更接近，同时仍能区分不同距离。

这里把它叫“语义表示”，是因为它被训练为表达任务相关内容，而不是保存整张图片的每一个像素。但它不是一句文字描述，也不是“距离10米、天气晴、车辆红色”这种可直接阅读的字段表。更准确地说，它是**学习得到的低维任务特征**；单个数的含义没有被人工逐项指定。

控制器将语义表示转成动作。它接收这32个数和另外提供的当前速度。比如两张图片都表示相近的距离，但车辆一个快、一个慢，需要的制动可能不同，因此速度必须一起输入。PPO训练让动作网络学会：这种观测下，哪种动作更有利于后续安全到达目标。

所以这条流程中，“理解画面”和“决定动作”是不同职责。DINO和投影提供信息，PPO利用信息决策；不是DINO直接给出刹车指令，也不是先把latent解码成距离，再交给PPO。

例子

假设车辆距障碍物约10米、当前速度2米/秒，摄像头看到一张雨天前车图像。这只是说明流程的场景，不是新增实验结果。

图片首先经过DINO和投影，成为32维表示。系统把速度2接在后面，得到33维观测。训练好的动作网络读取这一观测，给出一个制动动作。如果示例动作是1.2，当前解析模型会把下一速度计算为1.94米/秒；下一步再使用新的观测继续决策。

如果天气或车辆颜色改变，像素会改变，DINO特征也可能改变。我们通过多外观数据和对齐训练，希望在物理状态相同时，最终32维表示不要因为无关外观变化而过度改变。因此控制器可以把注意力放在任务所需信息上。

这仍是训练目标，不是自动保证。若某种外观使表示发生较大变化，PPO动作仍可能改变；后文的代理误差与SBC分析就是要检查这种变化是否影响安全。

从训练顺序看，先训练投影与代理表示，再固定表示训练PPO；在当前保留的控制与安全阶段，DINO、投影和q都不继续更新。因此，当前框架不是一次把图像编码器、PPO和SBC全部端到端联合训练成功。

训练时这条链怎样建立起来

训练后的运行过程是“输入图片，得到动作”。训练过程还多出一个关键步骤：根据错误或任务成绩，反过来修改网络参数。参数就是网络内部可调整的数字，修改它们会改变下次的输出。

当前这条链按以下顺序建立，不是三个阶段同时训练：

```
阶段一  学会提供信息
带距离标签的图片 → DINO特征 → 图像latent
同一距离          → q        → 代理latent
                       ↓
          比较两者差异、检查距离信息和外观一致性
                       ↓
          调整投影、q和训练用的辅助距离网络

阶段二  学会采取动作
固定好的图像latent或代理latent + 当前速度
                       ↓
                PPO随机尝试动作
                       ↓
          模拟车辆运动 → 下一状态和奖励
                       ↓
          收集一批经历，调整PPO动作和奖励价值网络

阶段三  按原SPVC做安全调整
固定前端和已训练的控制器 → 学习、检查SBC
                       ↓
          按原流程交替调整SBC和控制器动作网络
```

```
                    ↓
        保存控制器与SBC，并复查这一对保存模型
```

三个阶段的反馈来源不同：第一阶段用“图片对应什么距离、两条表示相差多少”；第二阶段用“采取动作后任务做得好不好”；第三阶段用“屏障条件是否满足”。不能把它们统称为“PPO训练”，也不能把第一阶段的表示误差当成第二阶段的控制奖励。

阶段	会更新什么	保持不变什么	训练结束得到什么
语义表示学习	投影、q、辅助距离网络；后续另有q单独拟合	DINO主体；此时不训练PPO和SBC	图像与距离可映射到相近的32维表示
初始PPO学习	动作网络、奖励价值网络和动作探索参数	DINO、投影、q	能根据32维表示与速度采取动作的完整PPO
原SPVC安全调整	按原流程交替更新SBC与控制器动作网络	DINO、投影、q；不进行完整PPO价值与探索训练	对应的一对控制器与SBC及检查结果

例如，一张标注为10米的图片，在第一阶段用于教网络保留距离信息；到了第二阶段，它只是提供观测的一种来源。PPO并不知道“这张图的标准刹车动作是多少”，而是通过执行动作后的奖励学习。

二 两条路径

路径一 让车辆根据图片做动作

```
图片 → 32维图像latent → 加上当前速度 → 控制器 → 动作 → 下一状态
```

路径二 在离线计算中检查控制器

```
指定距离 → 32维代理latent → 加上相同速度 → 同一个控制器 → 下一状态
```

三 图像路径中的每个模块到底干什么

当前图像路径可以展开为：

```
CARLA采集的RGB图片
        ↓
DINO视觉网络
        ↓ 384个视觉特征数
投影网络
        ↓ 32个任务特征数，称为图像latent
与当前速度拼接
        ↓ 33个输入数
PPO控制器
```

```
            ↓ 一个加速或制动动作
车辆动力学
            ↓ 下一距离、下一速度
下一次观察和决策
```

第一个模块 DINO负责看图

DINO是已经在大量图像上预训练的视觉网络。可以把它理解为一个已有视觉经验的特征提取器：输入图片，输出一组描述图片内容的数字。

我们使用DINOv2的小型版本，输出384个数字。本项目不更新DINO主体参数，这叫“冻结”。它不是每次都重新学习如何识别图片。

但是，DINO预训练不等于它已经学会本项目的安全制动。它提供通用视觉信息，后面的网络还需要把这些信息变成适合当前任务的输入。

第二个模块 投影网络把通用特征变成任务信息

投影网络输入DINO的384个数，输出32个数。它是一个较小的神经网络，参数会根据本项目的数据进行训练。

为什么不是直接使用384个数？因为我们希望控制器使用更小、便于和物理代理对齐的表示，不必处理所有通用视觉信息。

这里的32维是当前实现选择，不是理论要求只能32维，也不是越小越安全。

第三个模块 当前速度从哪里来

控制器除了看图片表示，还需要知道车辆现在有多快。

当前实验直接提供物理状态中的速度，把这一个数接在32维latent后面，形成33维输入。“拼接”就是把两组数放在同一个输入数组里。

目前没有用DINO从图片估计速度，也没有为车速传感器误差建立单独保证。因此，不能把当前结果说成仅靠单张图片完成所有状态估计。

第四个模块 PPO决定动作

PPO是强化学习算法，用来训练控制器。训练后的控制器本质上是一个神经网络：

> 输入这33个数，输出一个动作。

动作范围为−3到+3。在当前模型中，正值表示制动，负值表示加速。动作会裁剪到允许范围，例如网络输出4，实际执行的上限仍是3。

PPO不是安全证明器。它根据任务奖励学习：完成任务会得到更好的评价，进入危险状态会受到不利评价。这样的学习可以得到好策略，但有限训练并不能证明它在所有情况下安全。

第五个模块 动力学计算下一状态

动力学是“车辆执行动作后怎样变化”的规则。

当前解析模型每步为0.05秒，距离和速度的变化可概括为：

```
下一距离 = 当前距离 − 当前速度 × 0.05
下一速度 = 当前速度 − 制动动作 × 0.05
```

速度还被限制在0到3米/秒。这是当前模拟计算规则，不是完整实车模型。

执行动作得到下一状态，再获取对应的下一次观测、继续决策，这种循环叫“闭环”。只计算一张图片对应的动作，不叫完整闭环实验。

目前很多闭环实验使用图片库提供观测、解析模型计算运动。图片来自CARLA，但不能因此称为完整CARLA车辆动力学验证。

四 代理路径怎么得到同样的latent

代理路径是：

```
已知物理距离d
        ↓
代理网络q
        ↓ 32个数，称为代理latent
加上当前速度v
        ↓
与图像路径共用的PPO控制器
        ↓
动作和下一状态
```

q是一个小型神经网络。它的输入是经过尺度处理的距离，输出32维表示。尺度处理是为了让数字适合网络计算，并不是另一个感知算法。

这条路径里没有生成图片，也不经过DINO。

q在这里是我们给网络取的名字，不是强化学习中表示未来收益的Q函数。

一个完整例子

假设我们检查距离10米、速度2米/秒。这是用于说明流程的例子，不是一条新的实验测量。

图像路径会读取该距离下的图片，经DINO和投影得到一组32个数，再加上速度2，送入PPO。

代理路径会直接把距离10送入q，得到另一组32个数，再加上相同速度2，送入同一个PPO。

于是我们可以比较三件事：

1. 两组32维特征相差多少。

2. 同一个PPO在两组输入下给出的动作相差多少。
3. 两个动作在相同状态下造成的下一速度相差多少。

例如，若两个制动动作分别为1.0和1.2，根据当前模型，下一速度分别为1.95和1.94米/秒，偏差为0.01米/秒。这里的动作数字只是计算示例。

五 两条路径怎样训练到一起

第一阶段 数据集准备

目前收集了24000张图片，包含2000条距离采样、4种天气和3种颜色，保持原SPVC图像采集方式。

“配对数据”指图片和真实距离是一一对应的。比如知道这张雨天红车图片是距离10米时采到的，而不是只知道它是一张车的图片。

同距离的不同外观也能组成一组。这样可以区分：

> 图像不同，是车辆距离变了，还是仅仅天气和颜色变了？

按距离块分组划分后，训练集14172张，验证集4908张，测试集4920张。同一距离块的外观不会被随意拆到不同集合。

训练集用来更新网络；验证集用来检查和选择改进；测试集留作方案固定后的最终评价，目前尚未编码和评价。

第二阶段 训练投影网络和q

这里指完成数据准备后的表示训练，对应前面总览的“阶段一”。这一阶段做的是“学控制器输入”，不是“学制动动作”。

一批训练数据怎样走过两个网络

每条数据包含图片、真实距离、天气和颜色。先将图片经过冻结DINO得到384维特征。由于DINO不更新，这些特征可以先计算并缓存，后面的训练直接读取，不必反复跑整个视觉网络。

随后，同一条数据同时进入两条分支：

```
图片的384维特征 → 投影网络 → 图像latent
图片对应的距离 → q网络     → 代理latent
                                  ↓
                        对两条latent进行比较

图像latent ─┐
            ├→ 同一个辅助距离网络 → 各自恢复距离 → 与真实标签比较
代理latent ─┘
```

辅助距离网络的作用类似检查员：看这32个数中是否真的保留了距离信息。它在训练时参加计算，但运行控制时不走这条分支，PPO仍直接读取latent。

什么输出会被判为需要改进

表示训练会把几类误差合成一个总罚分，专业上称为“损失”。当前实现主要包括：

训练检查	为什么要检查	会推动网络做什么
图像latent与q的latent相差多少	同一个物理距离的两条输入应能互相替代	让投影和q输出更接近
从两条latent各自恢复的距离是否正确	两条输出接近不代表保留了有用信息	让表示保留实际距离信息
同一距离的不同外观是否接近	天气和颜色不应无必要地大幅改变控制输入	降低同距离外观造成的变化
距离相差至少1米的图像表示是否被混在一起	不能把远处和近处都变成相同表示	保留不同物理状态的可区分性
输出数值是否过大	避免仅靠放大数值改变表示尺度	对过大输出施加较小约束

例如两条latent都输出零，它们之间的误差是零，但不同距离无法被恢复，远近表示也无法区分，因此不会只凭“两条很像”就判定训练成功。

同距离的不同天气怎样一起训练

当前R2在多外观基础表示上继续训练。每批选8个不同距离组，把每组的不同天气、颜色图片一起计算；通常同距离有12种外观，重复采样位置可能包含更多记录。

对一个距离组，先算这一组图像latent的平均值，再计算各张图片离该平均值有多远。这里平均值是训练时临时计算的组中心，不是人工指定的正确latent标签。

组内差异越大，罚分越大。它与原来逐图像对q的罚分一起使用，所以不是只把同组图像拉近，却不管它们和物理代理是否一致。

当前R2新增组内一致性项的权重为0.2。额外的“组中心直接对q”项权重为0，没有启用；逐图像对q的对齐项仍然存在。

误差怎样使网络真正改变

计算总罚分后，程序计算哪些参数变化能够降低罚分，再用优化器小幅调整参数。这称为“反向传播”，不是把图像倒着生成回去。

这一轮共同更新投影、q和辅助距离网络。DINO保持冻结，PPO和SBC尚不参与这个罚分。因此第一阶段学到的是更合适的表示，而不是直接根据制动奖励优化视觉网络。

更新后再读取下一批训练数据，重复计算和调整。在检查节点上使用验证集计算表示和距离误差，选择开发指标较好的模型；验证集不用于这一步的梯度更新，但参与模型选择，因此不是

独立最终测试集。

当前实际训练和保存了什么

当前R2从已有多外观版本继续，做3000次联合参数更新，学习率为0.0003；新增同距离组约束与原配对、距离信息及区分性约束一起训练。

随后固定投影，只对q尝试400次进一步拟合，让它追随已固定的图像表示。这一小阶段不再改变图像latent。

实际保存的最佳联合版本来自第2600次更新；q后拟合的最佳选择是第0次，即没有采用后面400次更新中的候选。尝试过更新不等于最后一定采用它。

保存内容包括投影参数、q参数、辅助距离网络参数，以及使用的特征统计量和距离尺度。这样下一阶段读取的是同一套输入坐标，不会重新随机生成一套latent。

这一表示版本称为R2，它是实验版本名。它在验证集上的双路径相对表示误差约7.63%，距离辅助读取平均误差约0.166米。这是表示阶段的结果；车辆会不会成功制动，还需要下一阶段的PPO训练和评价。

第三阶段 在新表示上训练PPO

把32维latent加上当前速度，形成了新的33维输入。旧控制器原来不是按这套输入坐标训练的，不能直接把latent塞进旧网络并假定它会理解。

因此重新训练了适配新表示的PPO。下面说明这一次PPO究竟怎样训练，以及它的成绩应该怎样理解。

PPO训练

PPO要自己在模拟环境中尝试动作：执行后，环境返回下一状态和奖励；它根据多步运动后的成绩，调整以后选择动作的方式。

同一张图片在不同速度下，需要的制动可能不同。因此训练单位不是一张孤立图片，而是一段“观测、动作、运动、奖励”的连续过程。

每个回合先选图像路径还是代理路径

一次回合是从一个起点开始，持续执行动作，直到成功、危险、其他终止事件或超过400步。

当前R2初始PPO使用约一半代理回合、一半图像回合。是每回合开始时随机选择，不是把两个latent相加，也不是每一步都换路径。

- 代理回合：当前距离经过q，得到32维代理latent，再加当前速度。
- 图像回合：随机选一种天气和颜色，整个回合保持这一外观；每步在该外观的训练图片中，查找距离最接近当前距离的图片，读取它的32维图像latent，再加速度。

两种回合都进入同一个PPO。这样控制器不只适应比较干净的代理输入，也会接触图片造成的差异。

图像latent可以预先计算后存起来。训练读取这些缓存，是为了不每次重复运行DINO，并不是跳过图像表示学习。

这一阶段DINO、投影、q全部固定，只训练PPO。否则控制器还没学会，输入坐标又变了，就难以判断哪个模块导致结果变化。

训练从哪里开始 与测试从哪里开始不同

最初20万步PPO的训练环境随机起点为距离5至16米、速度0至3米/秒。这使训练能接触中间状态和靠近危险区域的状态；其中有些随机起点本来就很难恢复，不能把训练中的所有失败都解释为从规定出发区任务失败。

正式任务的回放评价起点则是距离15至16米、速度2.5至3米/秒。我们在这两个范围各取20个数，组成20×20共400个起点。

近期8192步的联合训练分支又使用规定出发区起点。这与最初完整PPO不是同一训练设置，需要分开报告。

奖励

当前实现包括：

事件	如何计分	希望产生的行为
向目标靠近	距离减少量乘2	不要一直停在远处
每经过一步	扣0.001	不要无限拖延
到达5至6米且速度不超过0.5	一次性加2，结束回合	低速进入目标范围
距离不超过6米但仍太快	扣除超出0.5米/秒的速度乘3，结束回合	避免高速进入危险区域

例如距离15米、速度2.5米/秒时，根据模型，这一步会靠近0.125米，得到0.25的前进奖励，再扣0.001。若没有终止事件，该步奖励是0.249。这个例子说明奖励计算，不是在指定最佳动作。

动作影响下一速度，也会影响后续是否安全到达。PPO因此需要考虑后续多步成绩，不能只选择当前前进最快的动作。

“提前停在目标外”和“超时”有独立结果标签，不会因为没有进入危险区域就被统计成成功。奖励值不是安全概率，奖励变高也不必然意味着风险下降。

这一版PPO训练效果怎样

先看进入原SPVC之前的第09步完整PPO，使用经过对齐的R2表示。评价时使用确定性动作，也就是不用训练时的随机探索，选当前策略给出的动作均值并裁剪执行。

评价输入	回合数	成功	不安全	超时
q代理latent	400	400	0	0
原天气的3种颜色	1200	1200	0	0
低太阳角度的3种颜色	1200	1200	0	0
阴天的3种颜色	1200	1200	0	0
雨天的3种颜色	1200	1200	0	0

合计5200个回合全部成功，但它们是同一组400起点在13种输入条件下重复评价，不是5200个全新的独立场景。图像回合仍用训练图片库最近距离回放。

代理回合平均约81.1步结束，各图像外观平均约78.1至80.2步。一步是0.05秒，对应约4秒的模型运动时间，不是代码执行耗时。

前一版R1表示下，相同12外观共4800图像回合有10次不安全；R2表示重训PPO后降到0。这个结果支持表示对齐改进和对应控制器重训有效，但不是保持PPO权重不变、只换表示的单因素实验。

为什么还检查图像和代理的动作差

完整回合成功，不能说明两条路径在每一个中间状态都一致。因此另外拿图像本身的真实距离，加上多个速度，分别输入图像路径和代理路径，比较动作。

第09步PPO的训练图像组合平均动作差约0.01985，验证图像组合约0.11959；验证最大动作差仍约4.339。动作差是制动减速度

验证集这147240个非终止图像与速度组合中，没有观察到“图像动作导致下一步危险、代理动作却没有”的情况。但这是单步开发检查，不是完整验证图像闭环，更不是所有连续状态的保证。

训练图片回放查图平均距离差约0.049米、最大约0.378米。它说明回放图片不一定恰好对应模型当前位置。因此，验证图像使用自身标签进行精确单步比较，没有对稀疏验证图片库强行做最近邻完整回合，避免把查图错配误当感知性能。

这一阶段可以汇报的成果是：**新32维表示已经能支持一个完成当前制动任务的PPO，而且同时适应图像和代理输入。** 尚不能汇报为未见图片和连续状态全部安全。

第四阶段 接入原SPVC的控制器与SBC训练

原SPVC后端的主要逻辑仍是：学习一个控制器，再学习和检查对应的SBC；若需要，再按原流程调整控制器与SBC。

这里有一个重要区别：完整PPO训练完成后，原SPVC取出它的动作网络，再围绕屏障条件调整这个动作网络。它不是继续用完整PPO算法采回合、更新奖励价值网络和动作分布。

原后端控制器更新主要希望动作满足屏障转移要求，同时不要过分偏离原来训练好的动作。可以理解为：先用强化学习学会开，再尝试按安全条件修正动作。DINO、投影和q保持固定，SBC按原方法交替训练。

因此，最终保存的控制器“来自PPO”，但它的最终动作权重经过原SPVC更新。不能把第09步完整PPO的5200回合成绩，直接当作最终保存控制器已经重新跑过的5200回合成绩。

最终保存控制器另做了12外观乘9个规定初始状态的108回合评价，全部成功，平均约76.35步、回报约20.94。该评价比第09步的400起点每外观更小，不用于声称全面优于原PPO。

“沿用原SPVC后端”指保留原训练和检查方法，并不指旧PPO权重一个数字都没变。PPO输入已变为33维，对应控制器需要适配并更新。

当前最小替换版本复用原仓库的后端训练、损失、噪声和网格规则。“损失”就是训练时对不符合要求的输出计罚，网络通过调整参数来减小罚分。这个版本没有另接运行时动作优化层或统计校准模块。

六 SBC放在哪里 它检查什么

SBC不是控制器后面串联的一个动作网络

SBC在这里是一个输入物理距离和速度、输出一个数的网络：

```
距离d、速度v → SBC → 一个屏障值B
```

它不读取32维latent，也不把latent进一步变成动作。

SBC要区分初始区域和危险区域

初始区域是车辆开始任务的状态范围：距离15至16米、速度2.5至3米/秒。

危险区域由原任务定义：距离不超过6米、速度仍大于0.5米/秒。成功则是进入距离5至6米的目标区域且速度足够低。

SBC希望在初始区域取较小值，在危险区域取较大值。白话上，这是让“正常出发”和“进入危险”在账本里明显分开。

但仅仅分开这两类状态还不够。车辆从初始状态经过很多步以后，是否会进入危险区域，还取决于中间的动作和运动。

SBC怎样和PPO发生关系

对一个待检查状态，当前分析流程是：

```
指定当前距离和速度
        ├→ SBC计算当前B值
        │
        └→ q生成代理latent
```

```
→ PPO给动作
→ 动力学与规定噪声得到下一状态
→ SBC计算下一B值
```

然后比较当前和下一状态的屏障值是否满足要求。

因此，PPO影响下一状态，下一状态影响SBC检查。它们通过车辆转移联系起来，而不是简单按“PPO→SBC→动作”串联。

为什么还要考虑噪声

如果下一状态完全确定，每个当前状态对应一个后继即可。原SPVC考虑的是带规定噪声的转移，需要分析不同噪声造成的后继，以及相应的期望关系。

“期望”就是按假设概率分布求平均，不等于要求每一种最坏扰动都下降。

SBC中的“S”表示随机性。对于这种分析，噪声范围和分布必须有依据；不能只因为代码里存在噪声项，就认为任何感知误差已经被覆盖。

当前保留了原代码的噪声假设。但DINO图像与q的实际差异，并没有自动被证明属于该噪声模型。

网格检查

距离和速度能取无限多个连续值，程序不能逐个穷举，所以先选取一批距离和速度组成网格，在网格上计算条件，并使用原代码的区间和补偿流程。

例如把距离分100份、速度分100份，共得到10000个网格状态。网格状态不是10000张图片，也不是10000次车辆任务。

原代码还会筛选一个与屏障值有关的区域进行下降检查。当前保存模型复查实际筛选了82个状态，其中1个硬违反；原代码按总网格1/10000计算，满足其不超过0.1%的容许规则。

因此“通过原代码检查”不是“所有连续状态零违反”。这两个说法不能互换。

七 换成latent后为什么误差仍然是关键

把框架放在一起，现在的关系是：

```
实际决策：图片 → DINO与投影 → 图像latent → PPO → 实际图像路径的下一状态
                                              ↑
                                    需要衡量并覆盖两路径差异
                                              ↓
离线分析：距离 → q → 代理latent → 同一个PPO → 代理路径的下一状态
                                                        ↓
                                                SBC检查转移是否满足条件
```

这里“实际图像路径”表示使用采集图像的路径，不表示已经完成实车实验。

现在SBC分析主要沿下方代理路径。要把它的结论用于上方图像路径，就必须处理两条路径之间的差异。

latent很接近 为什么动作仍可能差很多

控制器是一个非线性网络。某个输入位置附近，两个latent虽然数值接近，却可能位于不同动作决策一侧，导致制动差异较大。

因此不能只报告latent误差，还要报告动作差异和物理后继差异。

也不能只报告平均值。大多数时刻误差小，但靠近危险边界时恰好出现一次大误差，仍可能影响安全。

环境差异现在进入了哪里

天气和颜色已经进入训练图片；它们会影响图像latent，进一步影响PPO动作和下一状态。

同距离对齐希望弱化这些外观差异。但天气在本实验中只通过图像影响决策，不会自动让道路变滑、制动能力降低。后者属于动力学误差，需要另建模型或数据。

当前q只输入距离，也没有生成所有环境对应的不同latent。它输出该距离对应的一种近似表示，而不是已经包含全部环境变化。

还缺的保证是什么

我们需要明确一个范围：

> 对规定场景和状态，真实图像导致的动作相对q动作可能向上或向下偏多少？这个范围能够以什么依据覆盖未来样本？

之后将这个动作范围传入动力学，得到可能的下一状态，再检查SBC条件。这样安全分析才真正把图像与代理的差异算进去。

目前“观测到的动作范围”和“拟合出的误差区间”已做过开发实验，但后者仍漏掉一部分验证动作，不是合格的未来覆盖保证。

这不是要求latent逐个数字完全相等，而是要求：剩余差异进入安全分析，且分析仍然可行。

八 这个框架目前做到了什么

两条latent路径确实比早期更一致

验证集上，仅用单一外观训练时，双路径相对表示误差约78.89%；使用全部外观后降到8.70%；增加同距离外观对齐后为7.63%。

相对误差是“两个表示的差异相对表示本身变化幅度有多大”，不是不安全概率。7.63%不能读成车辆有7.63%的失败概率。

这些结果支持多外观数据和对齐训练有效，但两条路径仍不完全相同。

新代理的平均物理偏差比旧链更小

在相同4908张验证图像和对应状态下，得到：

比较内容	原cGAN相关链条	当前DINO链条	白话含义
图像距离平均绝对误差	2.668米	0.166米	当前表示更准确地保留距离信息
代理与图像造成的下一速度平均差异	0.07496米/秒	0.00604米/秒	代理更接近图像路径实际给出的运动结果

“距离误差”通过各系统的距离读取模块测量；当前辅助距离模块不因此成为PPO运行输入。

“下一速度差异”使用各系统自己的图像动作作参照，并按相同解析模型计算；不是实车下一速度测量。旧链使用仓库基础控制器，尚不是严格确认的原论文最终模型对照。

下一速度最大偏差约从0.215增至0.236米/秒，少数最差情况没有随平均值一起改善。这部分正是安全分析需要覆盖的尾部。

另外，共同32维评价空间中，新代理平均差异约0.0475，旧生成代理约0.9562。但评价器与本项目训练有关，旧图还存在低分辨率和灰度化，因此更适合说明“当前任务表示的一致性改善”，不能当作无偏的通用生成质量结论。

控制闭环和原SPVC后端已经接通

当前保留控制器在12种外观、9个规定初始状态，共108个确定性图片库回放回合中全部成功。

这说明已选的输入表示可以支持控制任务，不只是两个表示在数值上接近。它仍是有限初始范围的开发评价，不是所有状态和新图片都安全。

最终保存的PPO和SBC一起复查，通过原代码容许违反阈值，原代码概率读数约95.62%。

这个数字属于原代码口径。它还不能直接解释为真实图像车辆具有95.62%的可靠安全到达保证，因为图像与q差异的保证链尚未补齐。

九 CP尝试做了什么 为什么没有直接接入当前保证

CP在这个框架中负责什么

CP是Conformal Prediction，通常称为保形预测。这里把它用于统计校准：模型和计算规则固定后，用另一批独立轨迹估计一个阈值，再讨论新轨迹超过该阈值的概率。

“校准”不是再训练一次DINO或PPO，也不是CP自动替车辆刹车。它改变的是我们能够对误差或安全事件作出的统计说明，不是直接改变动作。

我们尝试过两种不同用途：第一种校准图像路径与代理路径的物理后继误差，再交给SBC验证；第二种直接校准一整条轨迹的屏障评分。两种方法的分数和结论不同，不能混在一起。

第一次尝试 给两条latent路径的误差划一个范围

这一尝试想补上前文提到的缺口：SBC沿q代理路径计算，但实际图像路径可能给出不同动作。我们希望先知道这个差异造成的下一状态偏差有多大。

冻结当时的R2表示和第09步完整PPO，按原SPVC全屏截图协议收集459条独立校准轨迹，每条最多400步。在每一步的同一个物理状态，分别计算图像动作和q动作，再用相同解析动力学计算两个后继。

当前动力学中，动作直接改变下一速度，不改变这一小步的下一距离。因此该实验的距离后继差为0，主要校准下一速度差，而不是直接校准32维latent之间的距离。

先对每条轨迹取“整条过程中最大的下一速度绝对偏差”，得到459个轨迹分数，再据此确定阈值：

```
同一当前状态
    ├→ 图像latent → PPO → 图像动作 → 图像路径下一速度
    └→ q latent  → PPO → 代理动作 → 代理路径下一速度
                                        ↓
                                比较两种下一速度的差
                                        ↓
                              每条轨迹取最大绝对差，再校准
```

得到的阈值约为0.150685米/秒，程序将其保存为对称速度误差范围，即约−0.150685至+0.150685米/秒。

该轮设定目标覆盖率99%、置信水平至少99%；程序给出的置信度为99.0079%。两个百分比的含义不同：覆盖率关注未来同分布轨迹有多大比例落在范围内；置信水平关注有限校准样本对这个覆盖结论的可靠程度。459是按这一统计要求确定的样本量。

这依赖校准轨迹与未来轨迹独立同分布、模型和协议固定、数据未被挑选等条件。“同分布”可以通俗理解为仍按同样的起点、外观和采集规则产生新任务。

所以，第一次得到的是**有条件的误差覆盖结果**，不是车辆安全结果。误差落在范围内时车辆是否安全，还要由后续SBC分析回答。

为什么把这个范围交给SBC后失败了

随后把范围内任意正负速度误差都作为允许扰动，训练鲁棒SBC。所谓“鲁棒”，这里是要求误差在规定范围里怎样变化，都能满足相应条件。

结果，8910个待检查状态中有8884个下降条件违反。为区分“网络没训练好”和“要求本身不可能”，又做了400个规定起点的压力测试：

每一步额外加到速度上的误差	安全终止	进入危险
固定取负端点−0.150685	400	0
始终为0	400	0
固定取正端点+0.150685	0	400

原因很直接：最大制动动作3，每步最多降低0.15米/秒；但误差范围允许每步再加回0.150685米/秒。若每一步都发生这个正误差，连最大制动也无法抵消，车辆会持续进入危险。

这不表示459条真实校准轨迹都失败了。它说明把轨迹误差变成“每步可任意重复极值”的范围，加入了比原来真实时序更强的最坏情况。CP对实际轨迹误差的覆盖，可以成立；要求对该范围内所有人为扰动序列都安全，则是另一个更强的问题。

之后还尝试按动作分12档，分别缩小误差范围。结果最大制动档的正误差仍约0.147901米/秒，持续施加正端点时，400个起点仍全部进入危险。仅按动作分档没有解决这个问题，因此没有继续用这条全局误差盒路线反复增加训练轮数。

第二次尝试 不构造任意扰动序列 直接校准整条轨迹

另一条尝试保留真实轨迹的先后关系，而不是把每一步误差都独立任意选择。

先用开发轨迹训练一个场景SBC，让正常出发和安全轨迹上的值较低，危险区域值较高。经过三版改进，最后一版初始和危险区域样本已分开，但验证转移仍有131/7453个动态违反，因此并未获得原SPVC那种所需状态域上的屏障转移结论。

随后冻结这一版屏障、当时的第09步PPO和评分规则，预先登记，再收集新的一批459条独立校准轨迹。这不是拿训练过SBC的旧轨迹重新充当独立校准集。

一条轨迹的评分定义为：

```
轨迹评分 = 起点屏障值 + 沿途所有正向增量之和 − 1
```

“正向增量”就是下一步屏障值比当前大了多少；若下降，则这一项按0计。把所有上涨累加，是为了不让后面一次下降抹掉前面出现过的上涨。

评分使用明确的停止处理：安全终止时屏障值按0，危险时按1。按这套定义，一条到达危险的轨迹，其评分不能为负。因此，若能够统计保证新轨迹的评分为负，就能进一步限制发生危险的概率。

该轮结果是：

项目	结果
新独立校准轨迹	459条，全部记录为成功
校准评分阈值	约−0.999808

项目	结果
目标覆盖率	99%
程序计算的置信度	99.0079%
程序状态	trajectory_sbc_cp_safety_pass

这里的负阈值是轨迹评分，没有米或米/秒的单位，与第一次0.150685米/秒的误差半径不是同一个量。

在冻结模型、独立同分布、轨迹及停止记录合规等前提下，这一结果支持**登记场景分布下、最多400步内不进入危险状态的统计结论**。它不是所有连续状态都满足屏障条件的证明，不是跨地图保证，也不是无限运行时间的保证。“避免危险”也不自动等于“必定完成目标”，尽管这459条实际结果均记录为成功。

还有一个需要诚实对照的地方：459条全成功本身就能用直接的成功/失败统计给出可靠性界限。因此，不能只因加入了SBC评分和CP，就说比直接统计成功率必然更强；屏障评分的额外价值还需要证明。

为什么当前框架仍然说安全保证没有完整补齐

第二次确实有一个正面的条件统计结果，不应该遗漏。但它绑定的是**当时第09步PPO、轨迹场景SBC第三版、对应截图与采样协议**。

后来接原SPVC后得到的minimal最终控制器，以及最新NBC联合训练的候选控制器，动作权重已经变化；当前保留的原SBC也不是那次轨迹CP使用的场景SBC。它们不能直接继承旧CP阈值和99%统计结论。

这与“当前保留模型通过原代码检查、读数约95.62%”是两组不同模型与不同口径的结果。不能把一个版本的CP数字接到另一个版本的控制器后，画成已经完成的统一安全链。

因此，CP并非没有做，也不是完全失败：

- 误差校准做成了，但把它扩成每步任意扰动后，鲁棒安全要求不可行。
- 冻结轨迹SBC后的统计校准得到正面结果，但只适用于那次登记的固定模型与分布。
- 当前minimal保留框架尚未得到与自身完全匹配的新CP结论，CP也不是当前运行时的动作修改模块。

如果继续这条方向，应先选定最终模型和要保证的事件，冻结后再用新独立轨迹校准；不能一边根据校准结果改模型，一边沿用同一批数据称作独立保证。

以上CP实验的完整配置和历史记录见完整报告第17至19章。服务器结果分别位于 results/dino_expanded_v1/14_transition_contract_spvc_v1、15_cp_robust_sbc_v1、15b_cp_box_feasibility、16_action_conditional_diagnostic 和 21_trajectory_sbc_cp_v1。

十 最近的联合安全训练在主框架中是什么位置

这一部分是单独实验分支，不是上面保留框架已经成功的新组成部分。

我们参考Model-Free NBC论文，尝试让控制器不仅学习完成任务，还学习避开屏障条件违反较大的动作。

NBC也是一个学习出来的屏障函数，但采用了与原随机SBC不同的形式。它用图像路径转移样本学习，并为PPO提供安全训练反馈。它不是接在动作后强制修改动作的模块。

训练反馈怎么影响PPO

采样一个动作，执行后得到下一状态。NBC判断这个转移违反了多少；训练时降低高违规动作的相对选择概率。

进一步尝试五步反馈，检查短期连续几个状态，而不只看立即下一步。

本分支保持DINO、投影和q固定；原SBC也没有被覆盖。更新对象是候选PPO和新NBC。

这一次才又使用完整PPO训练：将保留控制器的动作权重放入完整PPO中，奖励价值网络和探索设置来自第09步PPO，重新建立优化器。它不是原SPVC训练状态的原封不动恢复，所以作为独立候选分支记录。

到目前为止有什么结果

每组只做8192步的小预算实验，比较带安全反馈和不带安全反馈的普通PPO。

有限108回合测试仍全成功，但用同一个冻结屏障检查294480个验证图像和速度组合时：

- 原控制器不继续训练：13863个动态违反。
- 单步联合并保留旧约束：14583个。
- 五步联合并保留旧约束：14617个。
- 普通PPO对照：14508个。

“动态违反”是不满足屏障转移条件，不是车辆碰撞次数。这组结果没有显示安全训练优于原控制器或普通PPO。

确定性回放成功，也不等于随机探索安全。最新五步组在随机训练中，109个已结束回合有94次成功、15次不安全；108回合评价则不使用随机探索。两组数字来自不同设置，不能只展示评价全成功而省略训练中的失败。

候选NBC更新还会损害旧条件，因此加入检查和退回机制。后续四次候选更新全部被退回，说明保护旧结果做到了，但合格的新NBC还没有学出来。

所以当前保留的主框架没有切换到这些联合候选。

十一 当前到底卡在哪里 下一步只做什么

当前已经完成的部分

图片到latent、距离到代理latent、共享PPO、新表示训练、控制闭环、原SBC接入以及最终保存模型复查，都已经完成。

两条路径的平均误差比旧链小，也已有配对开发数据支持。

当前尚未完成的部分

没有完成的是：对新的真实图像路径，建立“误差被覆盖，而且对应安全条件成立”的完整结论。

另外，近期联合安全训练没有取得正面安全收益，不能把它算作已完成的改进成果。

接下来的两个问题

对于保留框架，优先解决图像与代理误差覆盖：先固定控制器，建立能包住图像动作差异的合理范围，再判断SBC是否还能满足条件。范围若有漏覆盖，不能因为检查通过就使用；范围若宽到条件根本不可能满足，也不能继续盲目训练。

对于联合训练分支，先检查“已成功停止”和“进入危险”的最后一步应怎样计入训练。目前一些成功末步被处罚，一些危险末步却没有正违规，说明屏障反馈还不是失败事件的准确替代。先核对这一点，再只改一个处理方式做同预算对照，不直接加大训练量。

这两个问题相互关联，但不是同一个任务。不能用“PPO继续训练了”代替“误差已经覆盖”，也不能用“误差平均更小”代替“屏障已经证明安全”。